

Introduction

If you've spent any time inside n8n, you've probably hit a moment where an expression that looked perfectly correct returned nothing, or threw an error that made no sense given the data you thought you had. Nine times out of ten, the root cause isn't your logic — it's a misunderstanding of the *shape* of the data flowing between nodes.

n8n doesn't pass a single object from node to node. It passes an array, always, no matter how many records are involved. This isn't an implementation detail you can ignore — it's the contract every node in the platform agrees to follow. Understanding it is the difference between confidently building multi-step workflows and randomly changing things until an error goes away.

This matters beyond n8n specifically. The same pattern — collections of records, each with a stable identity, flowing through a pipeline of transformations — shows up in ETL tools, message queues, and plenty of backend systems you'll touch as a developer. Learning to read it here builds an instinct that transfers directly to production code.

Problem Overview

Here's the core question this lesson answers: when node A finishes and node B starts, what exactly gets handed over?

The answer is a list. Every node in n8n receives and returns an array, where each element in that array is called an **item**. Each item is a small object with (at minimum) one key: `json`, which holds the actual data — the fields you care about, like an email address or an order ID.

Even when there's only one record moving through your workflow, it still arrives as an array with exactly one item inside it. There is no special "single object" mode. A workflow pulling one row from a database and a workflow pulling ten thousand rows both use the exact same shape — just with a different number of items in the array.

This uniformity is deliberate. It means every node can be written once, to operate on "a list of items," and it will work correctly whether it receives one record or one million.

Example

Imagine a node that fetches new signups from a form. It pulls back a single signup and hands it downstream like this:

```
[
  {
    "json": {
      "email": "jordan@example.com",
      "signed_up": true
    }
  }
]
```

Notice the outer square brackets — that's the array. Inside it, one object — that's the item. Inside the item, the `json` key — that's where your actual fields live.

Now compare that to three signups pulled at once:

```
[
  { "json": { "email": "jordan@example.com", "signed_up": true } },
  { "json": { "email": "amara@example.com", "signed_up": true } },
  { "json": { "email": "lee@example.com", "signed_up": false } }
]
```

Same structure, just more items in the array. Nothing about the shape changed — n8n never merges these into one blob with three names crammed into it. Each record keeps its own identity as a separate item, the same way a spreadsheet keeps each customer on its own row rather than jamming three names into a single cell.

Intuition

The way to build the right mental model isn't to memorize "arrays and json keys." It's to notice what problem this shape solves.

Any workflow tool needs to handle a variable number of records without changing its internal logic depending on that number. If a node's code had to special-case "one record" versus "many records," every single node in the platform would need branching logic, and every integration would be twice as likely to break silently the day your data source returns 2 rows instead of 1.

The array-of-items shape sidesteps that entirely. A node that transforms an email field doesn't need to know or care whether it's looking at 1 item or 10,000 — it just runs the same operation on every item in the array. This is the same reasoning behind why database query results, CSV parsers, and API list

endpoints almost always return arrays rather than a single object that happens to sometimes contain a list.

Once you see that items are the unit of data — not "the payload" as a whole — reading n8n's UI stops being guesswork. The Input and Output tabs on any node are just showing you "here's the array that arrived" and "here's the array that left." Comparing them tells you exactly what that node did.

Brute Force Solution

There isn't really a "solution" to build here — this lesson is about reading an existing contract, not implementing one. But it's worth walking through the naive mental model beginners often start with, because it's exactly what breaks:

The naive assumption: "A node gives me one object with my fields on it." Under this assumption, you'd write an expression like `{{ $json.email }}` and expect it to always work, because you're picturing a single flat object.

Why it mostly seems to work at first: Many workflows really do carry exactly one item most of the time, so `$json.email` resolves fine — until a node upstream starts returning multiple records, or a node accidentally strips the `json` key, and the illusion breaks.

Disadvantages: - No error message tells you your mental model is wrong — n8n just returns empty results. - You can't reason about multi-item workflows (loops, batch processing, splitting) at all, because your model has no concept of "more than one." - Debugging becomes guesswork instead of a lookup, since you never check Input/Output tabs — you just re-run things and hope.

Time and space complexity: Not applicable in the algorithmic sense, but the "cost" here is debugging time — unbounded, because you're searching for a bug without the right mental model to localize it.

Optimal Solution

The optimal approach is simply to internalize the real contract and use the tools n8n gives you to inspect it directly.

Step 1 — Accept the shape. Every node's data is `Array<{ json: {...}, ... }>`. Not "usually an array" — always.

Step 2 — Use Input and Output tabs deliberately. Click any node. Above its data panel are two tabs:

Tab**Shows**

Input The array of items that arrived into this node

Output The array of items that left this node

Flipping between them tells you precisely what a node added, removed, or transformed — you're diffing two arrays of items, not guessing.

Step 3 — Pick the right view for the job.**View****Best for**

JSON View Seeing exact structure, nested objects, precise key names

Schema View Quickly scanning fields across many items, spotting missing/extra fields at a glance

Schema View renders the same items as a table — one row per item, one column per field. For anything with more than a couple of records, it's dramatically faster to scan than raw JSON.

Step 4 — Know what "safe" transformation looks like. A node can add a field, remove a field, or transform a value (lowercase a name, drop a field). What it can never do is change the fact that the result is an array of items — that's fixed across the entire workflow, from the first node to the last.

Step 5 — Know the one way this silently breaks. If a node deletes the `json` key entirely (usually through a misconfigured Set or Code node), the item doesn't throw an error. It just loses its data and returns empty — no warning. If you ever see empty results with no explanation, check whether an upstream node stripped the `json` key rather than assuming your expression syntax is wrong.

Python Code

n8n's UI doesn't require you to write code for this lesson, but if you're using n8n's Code node (or reasoning about this pattern in your own scripts), here's the equivalent shape and a defensive read pattern in Python:

```
from typing import Any

def get_items() -> list[dict[str, Any]]:
    """Simulates what a node receives: an array of items, each holding
    its real data under a 'json' key."""
    return [
        {"json": {"email": "jordan@example.com", "signed_up": True}},
        {"json": {"email": "amara@example.com", "signed_up": True}},
        {"json": {"email": "lee@example.com", "signed_up": False}},
    ]
```

```
def extract_emails(items: list[dict[str, Any]]) -> list[str]:
    """Pulls the email field out of every item, skipping any item
    that's missing its json key instead of crashing on it."""
    emails = []
    for item in items:
        data = item.get("json")
        if data is None:
            continue # this item lost its json key upstream
        if "email" in data:
            emails.append(data["email"])
    return emails

if __name__ == "__main__":
    items = get_items()
    assert extract_emails(items) == [
        "jordan@example.com",
        "amara@example.com",
        "lee@example.com",
    ]
    assert extract_emails([{"json": None}, {"json": {"email": "a@b.com"}}]) == [
        "a@b.com"
    ]
    print("ok")
```

Code Walkthrough

- `get_items()` returns a `list[dict]` — this mirrors n8n's array-of-items shape exactly. Each dict has one key, `json`, holding the real data.
- `extract_emails()` iterates the array (the "courier"), and for each item pulls out `json` with `.get()` rather than `item["json"]`. This mirrors the real-world failure mode: if a node upstream stripped the `json` key, `.get()` returns `None` instead of raising a `KeyError`, so we can skip that item gracefully instead of crashing the whole batch.
- The function processes every item identically regardless of how many there are — one item or a thousand run through the same loop body, which is exactly the property that makes the array-of-items shape useful in the first place.
- The `__main__` block is a self-check: it asserts normal extraction works, and that an item with a missing/ `None` `json` key is skipped rather than breaking the run.

Dry Run

Take the three-item list from `get_items()`.

1. `data = item.get("json")` for item 1 → `{"email": "jordan@example.com", "signed_up": True}`. `"email"` is present, so `"jordan@example.com"` is appended.
2. Same for item 2 → `"amara@example.com"` appended.
3. Same for item 3 → `"lee@example.com"` appended.
4. Loop ends. `emails = ["jordan@example.com", "amara@example.com", "lee@example.com"]`.

Now the edge case: `[{"json": None}, {"json": {"email": "a@b.com"}}]`.

1. Item 1: `data = None` → `if data is None: continue` skips it, no crash.
2. Item 2: `data = {"email": "a@b.com"}` → appended.
3. Result: `["a@b.com"]` — exactly what the assertion checks.

Complexity Analysis

- **Time:** $O(n)$, where n is the number of items. Each item is visited exactly once, and dictionary key lookups (`.get` , `in`) are $O(1)$ on average.
- **Space:** $O(k)$, where k is the number of items that actually contain an email — bounded by n , since the output list can never be larger than the input.

These bounds are correct because there's no nested iteration over items (no item is compared against another item), and the output only grows when a valid email is found, never independently of the input size.

Alternative Solutions

A list comprehension does the same job more tersely:

```
def extract_emails(items):
    return [
        item["json"]["email"]
        for item in items
        if item.get("json") and "email" in item["json"]
    ]
```

This is functionally equivalent and still $O(n)/O(k)$. The explicit loop version is easier to extend if you later need per-item logging or error collection; the comprehension is more idiomatic Python for a simple filter-and-map. Neither is "more correct" — pick based on whether you expect this logic to grow more branches.

Edge Cases

- **Empty array input:** `get_items()` returning `[]` → `extract_emails` returns `[]`, no error. This mirrors n8n correctly: zero items is a valid, well-formed state, not an error condition.
- **Missing json key entirely:** Handled by `.get("json")` returning `None` and being skipped — this is the exact real-world bug described in the video (a node accidentally deletes the `json` key).
- **json present but missing the field you want:** The `"email" in data` check prevents a `KeyError` when a specific field, rather than the whole `json` object, is missing.
- **Very large item counts:** The linear scan holds up fine; the risk in real n8n workflows is memory, not this loop, since all items for a batch typically load at once.
- **Duplicate values across items:** Nothing here deduplicates — if two items have the same email, both are kept, matching n8n's behavior of never silently merging items.

Common Mistakes

1. Writing expressions assuming a single flat object instead of an array of items, then being confused when a workflow scales to multiple records.
2. Not checking the Input tab before assuming a bug is in the current node — often the bad data already arrived broken from the node before it.
3. Assuming JSON View and Schema View show different data — they show the exact same items, just formatted differently.
4. Overwriting the entire `json` object in a Set/Code node instead of merging into it, which accidentally drops fields the next node depends on.
5. Treating an empty result as an error state rather than checking whether the `json` key survived the last transformation.
6. Debugging by re-running the whole workflow repeatedly instead of comparing a single node's Input and Output tabs directly.

Interview Questions

- Why do you think a workflow engine chooses to always represent data as a collection, even for a single record?
- How would you design a system so that every processing step doesn't need to special-case "one item" vs "many items"?
- What are the tradeoffs of failing silently (returning empty) versus throwing an explicit error when expected data is missing?

- How would you write a function that processes a list of records where some may be malformed, without letting one bad record halt the whole batch?
- What's the difference between a schema-on-read and schema-on-write approach, and how does that relate to Schema View vs JSON View?

Similar Problems

- **Flatten Nested List Iterator (LeetCode 341):** builds the same intuition — treating a nested structure as a flat sequence of items to process one at a time.
- **Group Anagrams (LeetCode 49):** like grouping items by a shared field, similar to how you'd reason about batches of n8n items sharing a common key.
- **Design an Iterator over a List of Lists:** directly parallels iterating a list of items each wrapped in their own container, the same shape as n8n's array-of-items.
- **Merge K Sorted Lists (LeetCode 23):** reinforces the habit of treating "a collection of collections" correctly instead of flattening prematurely.

Key Takeaways

n8n always passes data between nodes as an array, and every item inside that array carries its real fields under a `json` key. This isn't incidental — it's the uniform contract that lets every node handle one record or a million with the same logic. Reading the Input and Output tabs on any node, and choosing Schema View when you just need to scan fields quickly, turns debugging from guesswork into a direct comparison. And the one failure mode worth remembering: if the `json` key ever gets deleted, the item doesn't error — it just goes silently empty, so that's always your first suspect when a workflow produces nothing.

Watch the Video

This lesson is easier to absorb watching it live inside a real n8n workflow — seeing the Input/Output tabs flip in real time makes the array-of-items shape click much faster than reading about it. Watch the full walkthrough here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around opening the hood on developer tools instead of just clicking through them. Rather than treating n8n, or any tool, as a black

box, each lesson digs into the actual data structures and mechanics underneath — the kind of understanding that makes you fast at debugging instead of guessing.

Call To Action

If this helped the array-of-items concept finally click, give the video a like on YouTube and subscribe so you don't miss the next lesson in the series. For deeper written breakdowns like this one, subscribe to the Substack newsletter. And if something in this explanation felt off, unclear, or like it's missing a piece — drop a comment. Lessons in this series get shaped directly by what viewers ask for.

Quick Review

What sits between every n8n node execution?

An items array — a JSON list, not a single object, passed from node to node.

Why an array, not one object?

Nodes can process multiple records at once; the array holds one entry per record.

How do items move through a workflow?

Each node reads the previous node's output items as its own input items.

Where do you inspect a node's data in the UI?

Input and Output tabs on the node panel, showing items before/after that node ran.

Schema view vs JSON view — when to use each?

Schema view shows field structure/types at a glance; JSON view shows the raw exact data per item.

What's the classic beginner mistake with items?

Assuming there's always exactly one item, so multi-item runs silently drop or mishandle extra records.

n8n Lesson 5: Understanding the data structure (items) (JSON items array, how data flows between nodes, viewing input/output per node) — free Python cheat sheet